16 EXCEPTION HANDLING • DETAILED SOLUTIONS

conceptual solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Conceptual Questions (Module 16: Exception Handling)

Solution to Q1 (Compile Errors vs. Exceptions vs. Undefined Behavior)

- **Compile-time errors**: Syntax or type/access violations rejected by the compiler prior to execution.
- **Synchronous runtime exceptions**: Runtime conditions reported explicitly via `throw` and caught via `catch`.
- **Hardware traps / Undefined Behavior**: Operations like raw `1 / 0` or `*nullptr` do **not** throw C++ exceptions; you must explicitly check preconditions (`if (den == 0) throw ...;`) to enter the C++ exception mechanism.

Solution to Q2 (Stack Unwinding & Constructor Exceptions)

- During **Stack Unwinding**, all fully constructed automatic (stack) objects in the exited `try` block and intermediate call frames are destroyed in **reverse order of construction (LIFO)**.
- If a constructor throws an exception before finishing, destructors are run for all **already-constructed base classes and member subobjects**, but the destructor for the partially constructed object itself is **not** called.

Solution to Q3 (`throw;` vs `throw e;`)

- `throw;` rethrows the original in-flight exception object without copying or slicing it.
- `throw e;` copies `e` using its static parameter type (`BaseException`), slicing off any `DerivedException` part!

Solution to Q4 (Basic, Strong, and No-Throw Guarantees & Destructors)

- **Basic**: No leaks; invariants remain valid, though state may have partially changed.
- **Strong**: Commit-or-rollback; state is completely unchanged if an exception occurs.
- **No-Throw (`noexcept`)**: Never throws; always succeeds.
- **Why destructors must not throw**: Destructors are called during stack unwinding while an exception is already active. If a destructor throws a second unhandled exception during stack unwinding, C++ cannot propagate two exceptions simultaneously and immediately calls `std::terminate()`.